

Índice del Repositorio: `web_integral` — Proyecto "El Zapatito"

Repositorio: PablitinGlez/web_integral **Descripción general:** Proyecto universitario de e-commerce para la tienda de calzado ficticia "Zapatín", desarrollado bajo metodología Scrum por un equipo de 5 integrantes. Incluye documentación de análisis/diseño, código fuente (frontend Angular + backend FastAPI/Python) y entregables académicos.

1. Documentación raíz

- `Documento General del Proyecto.pdf` — Documento maestro del proyecto.

2. `cliente/` — Carpeta principal de entregables

Contiene toda la documentación formal del proyecto dirigida al cliente (Zapatín) y el código fuente del sistema.

2.1 Gestión y equipo

Archivos sueltos dentro de `cliente/` sobre organización del equipo, forma de trabajar y acuerdos de colaboración.

- `integrantes.md` — Integrantes del equipo: Juan Pablo Gonzales Cuevas, Eliezer Isaí Cerecedo Florencia, Jocelyn Serrano Montaña, Jhonatan David Victoria López, Alberto Valerio Romero. Equipo cliente: **Zapatín**.
- `flujo_de_trabajo.md` — Protocolo de trabajo con Git Flow: ramas `dev - [nombre]` → `desarrollo` → `main`, con peer review y reglas de commits.
- `metodologia_elegida.md` — Justificación del uso de **Scrum** (sprints, backlog, ceremonias, auto-organización).
- `Acuerdo de ramificación.pdf` — Acuerdo formal de ramificación del equipo.

2.2 Analisis_Diseño/ — Documentos UML y modelado

Carpeta con los artefactos de análisis y diseño previos a la programación: diagramas UML que modelan el comportamiento y la estructura del sistema, más el diccionario de datos.

- `Diagrama_actividades.pdf`
- `Diagrama_casos_de_uso.pdf`
- `Diagrama_clases.pdf`
- `Diagrama_componentes.pdf`
- `Diagrama_flujo.pdf`
- `Diagrama_secuencia.pdf`

- `Diccionario_de_datos.pdf`
- `Modelado de base de datos.jpeg`

2.3 arquitectura_sistema/

Carpeta que documenta cómo está organizada técnicamente la aplicación (arquitectura por capas: presentación, lógica de negocio y datos).

- `Arquitectura.md` — Descripción escrita de la arquitectura por capas.
- `Arquitectura por capas.jpeg` — Diagrama visual de la arquitectura.

2.4 diseño_interfaces/

Carpeta con el manual de identidad visual de la marca, usado como guía para el diseño del frontend.

- `Manual_Identidad_Zapatito.pdf` — Manual de identidad de marca (paleta de colores, iconografía, logotipo, estilo visual).

2.5 implementacion/

Carpeta que justifica las decisiones técnicas tomadas para construir el sistema.

- `stackjustificacion.md` — Justificación del stack tecnológico elegido (por qué Angular, por qué FastAPI, por qué Supabase, etc.).

2.6 manual_tecnico/

Carpeta dirigida a desarrolladores/soporte técnico, con la explicación de cómo funciona y se mantiene el sistema.

- `Manual_Tecnico_El_Zapatito.pdf`
- `README.md`

2.7 manual_usuario/

Carpeta dirigida al usuario final, con instrucciones de uso de la tienda y el panel administrativo.

- `Manual_de_Usuario_El_Zapatito.pdf`
- `README.md`

2.8 metodologia/ — Artefactos Scrum

Carpeta con toda la evidencia del proceso ágil seguido durante el proyecto: planeación, seguimiento y retrospectiva de cada sprint.

- `Product_Backlog_El_Zapatito.pdf`
- `Sprint_Backlogs_El_Zapatito.pdf`
- `Sprint_Reviews_El_Zapatito.pdf`
- `Sprint_Retrospectives_El_Zapatito.pdf`

- `Burndown_Charts_El_Zapatito.pdf`
- `Historias_de_Usuario_El_Zapatito.pdf`
- `Incrementos_de_Software_El_Zapatito.pdf`
- `Roles.md`

2.9 codigo/ — Código fuente

Carpeta que contiene todo el código fuente del sistema, dividido en backend y frontend.

a) *backend/* — El "cerebro" del sistema (Python)

Es la parte que trabaja detrás de cámaras: recibe lo que pide la página web (por ejemplo, "muéstrame los productos" o "guarda este pedido"), lo procesa y guarda o consulta la información en la base de datos.

- `main.py` — Archivo principal que enciende el servidor y conecta todas sus partes.
- `requirements.txt`, `pyrefly.toml` — Lista de herramientas que necesita el proyecto para funcionar.
- `schema.sql`, `FINAL_REPAIR_SCHEMA.sql` — Archivos que definen cómo está armada la base de datos.
- `DATABASE_GUIDE.md` — Explicación de qué guarda cada tabla de la base de datos (productos, categorías, inventario, pedidos, usuarios, etc.).
- Scripts de apoyo: `seed.py`, `populate_inventory.py`, `repair_db.py`, `fix_supabase_db.py` — Sirven para llenar de datos de prueba o corregir la base de datos.
- `test.db`, `zapatito.db`, `repair_log.txt` — Archivos de datos y registros usados durante el desarrollo.

app/api/ — Aquí están las "puertas de entrada" que usa la página web para pedir o enviar información. Hay un archivo por cada tema del negocio.

- `addresses.py` — Direcciones de envío de los usuarios.
- `auth.py` — Inicio de sesión y datos del perfil.
- `brands.py` — Marcas de calzado.
- `categories.py` — Categorías de productos.
- `coupons.py` — Cupones de descuento.
- `inventory.py` — Existencias disponibles por talla.
- `metrics.py` — Datos y estadísticas para el panel administrativo.
- `orders.py` — Pedidos realizados por los clientes.
- `products.py` — Productos: buscar, filtrar y mostrar.

- `suppliers.py` — Proveedores.

app/models/ — Aquí se describe cómo son las tablas de la base de datos, en forma de código. Un archivo por cada tipo de información que se guarda.

- `address.py` — Direcciones.
- `base.py` — Base común que usan todos los demás archivos de esta carpeta.
- `brand.py` — Marcas.
- `category.py` — Categorías.
- `coupon.py` — Cupones.
- `order.py` — Pedidos.
- `product.py` — Productos.
- `product_variants.py` — Tallas y variantes de cada producto.
- `supplier.py` — Proveedores.
- `user.py` — Usuarios y sus roles (cliente o administrador).

app/schemas/ — Define qué datos son válidos al enviar o recibir información (por ejemplo, que un precio sea un número y no un texto). Hay un archivo por cada tema, igual que en `models/`.

- `address.py`, `brand.py`, `category.py`, `coupon.py`, `order.py`, `product.py`, `product_variants.py`, `supplier.py`, `user.py`

app/services/ — Funciones de apoyo que usan varias partes del sistema.

- `auth_service.py` — Maneja el inicio de sesión y la seguridad de las cuentas.
- `clouinary_service.py` — Sube y administra las fotos de los productos.

app/core/ — Configuración general del sistema.

- `config.py` — Guarda ajustes y claves necesarias para que la aplicación funcione.

app/database.py — Conecta el sistema con la base de datos.

tests/ — Pruebas para verificar que todo funcione bien.

- `test_discount_and_metrics.py` — Prueba que los descuentos y las estadísticas se calculen correctamente.

b) el-zapatico/ — Aplicación web (Frontend, Angular 21)

Es la interfaz que interactúa directamente con el usuario: por un lado la tienda pública (catálogo, carrito, checkout) y por otro el panel de administración interno. Está construida con **Angular 21** usando componentes *standalone* (sin `NgModules`) y *lazy loading* por ruta (`loadComponent`), lo que significa que cada pantalla se carga bajo demanda solo cuando el usuario navega a ella, mejorando el tiempo de carga inicial. Las rutas están organizadas en dos grandes bloques dentro de `app.routes.ts`: todo lo que cuelga de `'` (tienda + auth) usa el layout `store-layout`, y

todo lo que cuelga de `/admin` usa el layout `admin-layout`. El estado (por ejemplo, el carrito de compras en `cart.service.ts`) se maneja con **signals** de Angular en lugar de RxJS puro, lo que es el enfoque moderno recomendado por el framework.

- `angular.json` — Configuración de build, assets y opciones del proyecto Angular.
- `package.json`, `package-lock.json` — Dependencias de Node.js y scripts (`ng serve`, `ng build`, `ng test`).
- `tsconfig.json`, `tsconfig.app.json`, `tsconfig.spec.json` — Configuración del compilador de TypeScript.
- `.editorconfig` — Reglas de formato consistentes entre editores.
- `.vscode/` — Configuración del editor (extensiones recomendadas, tareas, configuración de debug y MCP).
- `README.md` — Instrucciones estándar generadas por Angular CLI (cómo levantar el servidor, compilar y correr pruebas).

public/ — Archivos estáticos que se sirven tal cual, sin procesar.

- `logo.png` — Logotipo de la marca Zapatín, usado en el header/nav.
- `favicon.ico` — Ícono de la pestaña del navegador.

src/app/components/admin/ — Pantallas del panel administrativo, protegido para uso interno del equipo de la tienda. Cada carpeta es un componente independiente enfocado en gestionar una entidad del negocio.

- `admin-layout` — Estructura/menú general que envuelve todas las pantallas de administración.
- `dashboard` — Panel principal con métricas y resumen del negocio (ventas, pedidos, etc.), alimentado por el endpoint `metrics.py` del backend.
- `add-product` — Formulario para dar de alta nuevos productos.
- `products` — Listado y edición de productos existentes.
- `brands` — Gestión de marcas.
- `categories` — Gestión de categorías.
- `coupons` — Gestión de cupones/descuentos.
- `inventory` — Gestión de existencias por talla.
- `orders` — Seguimiento y gestión de pedidos realizados por clientes.
- `suppliers` — Gestión de proveedores.
- `settings` — Configuración general del panel administrativo.

src/app/components/store/ — Pantallas de la tienda pública, la parte del sitio que ve cualquier visitante.

- `store-layout` — Estructura general de la tienda (header, footer, navegación) que envuelve todas las páginas públicas.
- `home` — Página de inicio.
- `catalog` — Catálogo de productos con filtros (categoría, precio, género, marca, búsqueda).
- `product-detail` — Ficha de detalle de un producto específico (tallas, imágenes, precio).
- `checkout` — Proceso de compra: revisión del carrito, dirección y pago.
- `about` — Página "Nosotros" / información de la tienda.

`src/app/components/auth/` — Pantallas relacionadas con la cuenta del usuario.

- `login.component.ts` — Formulario de inicio de sesión.
- `register.component.ts` — Formulario de registro de nuevos usuarios.
- `profile.component.ts` — Vista y edición del perfil del usuario autenticado (incluye sus direcciones guardadas).

`src/app/components/privacy-policy/` — Página informativa con la política de privacidad de la tienda, requerida para cumplimiento legal/manejo de datos de usuarios.

- `privacy-policy.ts` — Lógica del componente.
- `privacy-policy.html` — Plantilla/maquetado.
- `privacy-policy.css` — Estilos propios de la página.
- `privacy-policy.spec.ts` — Prueba unitaria del componente.

`src/app/services/` — Capa de servicios de Angular (`@Injectable`) que hace las peticiones HTTP hacia la API del backend y centraliza la lógica de datos, para que los componentes no llamen a la API directamente.

- `address.service.ts` — Comunicación con el endpoint de direcciones.
- `auth.service.ts` — Login, registro, manejo de sesión/token del usuario.
- `brand.service.ts` — Consumo del endpoint de marcas.
- `cart.service.ts` — Carrito de compras manejado en memoria con *signals* (estado reactivo local, no persiste en base de datos hasta el checkout).
- `category.service.ts` — Consumo del endpoint de categorías.
- `order.service.ts` — Creación y consulta de pedidos.
- `product.service.ts` — Consumo del endpoint de productos (listado, filtros, detalle).
- `supabase-client.ts` — Cliente de conexión directa a Supabase desde el frontend (por ejemplo, para autenticación o storage, en paralelo al backend propio).

- `supplier.service.ts` — Consumo del endpoint de proveedores.

src/app/guards/ — Guardias de ruta de Angular: funciones que se ejecutan antes de entrar a una ruta para decidir si el usuario tiene permiso de verla.

- `auth.guard.ts` — Verifica que el usuario esté autenticado antes de dejarlo acceder a rutas protegidas (como `/admin` o `/profile`).

Núcleo de la app (`src/app/` y `src/`):

- `app.routes.ts` — Define el árbol completo de rutas de la aplicación: bloque público (`store-layout` con `home`, `catálogo`, `producto`, `checkout`, `about`, `login`, `registro`, `perfil`, `política de privacidad`) y bloque `/admin` (`admin-layout` con sus subrutas).
- `app.config.ts` — Configuración global de la app (`providers`, `router`, etc.).
- `app.ts`, `app.html`, `app.css` — Componente raíz de la aplicación (shell que contiene el `<router-outlet>`).
- `app.spec.ts` — Prueba unitaria del componente raíz.
- `src/main.ts` — Punto de arranque que hace *bootstrap* de la aplicación Angular en el navegador.
- `src/index.html` — HTML base donde se monta la aplicación.
- `src/styles.css` — Estilos globales del proyecto.

3. mi_proyecto/ — Documentación individual

Carpeta de documentación individual del alumno, fuera del entregable grupal (`cliente/`).

3.1 integrantes/

Información individual sobre el integrante del equipo.

- `README.md`

3.2 mi_problematika/

Documento donde se plantea la problemática abordada por el proyecto desde la perspectiva individual.

- `README.md`

Resumen técnico del stack

Capa	Tecnología
Frontend	Angular 21 + TypeScript
Backend	Python (FastAPI, estructura <code>app/api</code> , <code>app/models</code> , <code>app/schemas</code>)

Capa	Tecnología
Base de datos	Supabase (PostgreSQL)
Almacenamiento de imágenes	Cloudinary
Metodología	Scrum
Control de versiones	Git Flow (dev- * → desarrollo → main)

Resumen funcional (dominio de negocio)

Tienda de calzado con: catálogo público, carrito y checkout, gestión de usuarios/autenticación, panel administrativo (productos, marcas, categorías, proveedores, inventario por tallas, cupones/descuentos, pedidos, métricas y dashboard).